Simple practical-format answers containing aims, sample data, commands/programs, expected outputs, explanations and results.

Important: Replace /path/ with the actual file location and use your college lab's Hadoop, Hive and Spark configuration. Outputs are expected for the sample data; capture your own screenshots after running the commands.

BIG DATA

Practical Record – Answers for All 9 Test Use Cases

Practical 1: Hive Functions – Demo

Aim

Create a Hive table, load student marks and execute aggregate, string and date functions.

Sample CSV: student.csv

```
1,Monika,IT,90,2024-06-10
2,Priya,CSE,85,2024-07-15
3,Sanjay,IT,75,2024-06-20
4,Rahul,CSE,95,2024-08-12
5,Divya,IT,80,2024-07-25
```

Create database and table

```
CREATE DATABASE IF NOT EXISTS college;
USE college;
CREATE TABLE students (
 id INT, name STRING, department STRING,
 marks INT, admission_date STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

Load data and display

```
LOAD DATA LOCAL INPATH '/path/student.csv' INTO TABLE students;
SELECT * FROM students;
```

Aggregate functions

```
SELECT COUNT(*) FROM students; -- 5
SELECT SUM(marks) FROM students; -- 425
SELECT AVG(marks) FROM students; -- 85.0
SELECT MIN(marks) FROM students; -- 75
SELECT MAX(marks) FROM students; -- 95
```

String and date functions

```
SELECT UPPER(name) FROM students;
SELECT name, LENGTH(name) FROM students;
SELECT YEAR(TO_DATE(admission_date)) FROM students;
```

Change conditions

```
SELECT * FROM students WHERE marks > 80;
SELECT * FROM students WHERE department='IT';
SELECT AVG(marks) FROM students WHERE department='IT'; -- 81.6667
```

Result

All Hive functions were executed. The total count is 5, total marks 425, average 85, minimum 75 and maximum 95.

Practical 2: Advanced Hive – Partitioning and Bucketing

Aim

Create partitioned and bucketed tables and run filtered queries.

Create partitioned table

```
CREATE TABLE student_partitioned (
 id INT, name STRING, marks INT
)
PARTITIONED BY (department STRING)
STORED AS PARQUET;
```

Insert records into partitions

```
INSERT INTO student_partitioned PARTITION(department='IT')
SELECT id,name,marks FROM students WHERE department='IT';
INSERT INTO student_partitioned PARTITION(department='CSE')
SELECT id,name,marks FROM students WHERE department='CSE';
```

Query a partition

```
SELECT * FROM student_partitioned WHERE department='IT';
```

Create bucketed table

```
SET hive.enforce.bucketing=true;
CREATE TABLE student_bucketed (
```

```
 id INT, name STRING, department STRING, marks INT
)
CLUSTERED BY (id) INTO 4 BUCKETS
STORED AS ORC;
```

Insert and query bucketed table

```
INSERT OVERWRITE TABLE student_bucketed
SELECT id,name,department,marks FROM students;
SELECT * FROM student_bucketed;
```

Complex data type – ARRAY

```
CREATE TABLE student_skills (
 id INT, name STRING, skills ARRAY<STRING>
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
COLLECTION ITEMS TERMINATED BY '|';
SELECT name, skills[0] FROM student_skills;
```

Explanation

Partitioning stores data in separate partitions based on a column such as department, helping queries that filter that column. Bucketing divides rows into a fixed number of files using a hash of a selected column; it can help joins and sampling. Support and settings depend on the Hive version.

Result

Partitioned and bucketed tables were created and queried. Partitioning organizes data by column values, while bucketing distributes rows into a fixed number of buckets.

Practical 3: Spark Core – RDD Word Count

Aim

Use RDD transformations and actions to count words in a text file.

Input file: input.txt

```
big data is big
spark processes big data
```

PySpark program

```
from pyspark import SparkContext
sc = SparkContext.getOrCreate()
lines = sc.textFile('input.txt')
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word.lower(), 1))
counts = pairs.reduceByKey(lambda a,b: a+b)
print('Total words:', words.count())
print('Word counts:', sorted(counts.collect()))
sc.stop()
```

Expected output

Total words: 8 [('big', 3), ('data', 2), ('is', 1), ('processes', 1), ('spark', 1)]

Explanation

Transformations such as flatMap, map and reduceByKey create new RDDs and are evaluated lazily. Actions such as count and collect trigger execution and return results.

Result

RDD transformations and actions were used to calculate the word frequencies.

Practical 4: Spark DataFrame – Demo

Aim

Load a CSV file into a Spark DataFrame, inspect its schema and perform operations.

CSV input

```
id,name,department,marks
1,Monika,IT,90
```

```
2,Priya,CSE,85
3,Sanjay,IT,75
4,Rahul,CSE,95
5,Divya,IT,80
```

PySpark program

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, count
spark = SparkSession.builder.appName('StudentDF').getOrCreate()
df = spark.read.option('header',True).option('inferSchema',True).csv('student.csv')
df.printSchema()
df.show()
df.select('name','marks').show()
df.filter(df.marks > 80).show()
df.groupBy('department').agg(count('*').alias('students'),avg('marks').alias('average')).show()
spark.stop()
```

Expected aggregate

```
CSE: 2 students, average marks = 90.0
IT: 3 students, average marks = 81.6667
```

Explanation

A DataFrame is distributed structured data organized into named columns. printSchema displays column names and data types; select chooses columns, filter selects matching rows, and groupBy performs grouping.

Result

CSV data was loaded and schema, selection, filtering, grouping and aggregation were demonstrated.

Practical 5: Spark SQL

Aim

Register a DataFrame as a temporary view, execute SQL queries and save output.

PySpark program

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('SparkSQL').getOrCreate()
df = spark.read.option('header',True).option('inferSchema',True).csv('student.csv')
df.createOrReplaceTempView('students')
spark.sql('SELECT * FROM students').show()
spark.sql("SELECT name,marks FROM students WHERE marks >= 85").show()
spark.sql('SELECT department,AVG(marks) AS average FROM students GROUP BY department').show()
spark.sql('SELECT * FROM students ORDER BY marks DESC').show()
result = spark.sql('SELECT department,AVG(marks) AS average FROM students GROUP BY department')
result.write.mode('overwrite').parquet('output/student_average')
spark.read.parquet('output/student_average').show()
spark.stop()
```

Join query

```
If a second DataFrame named departments(department,hod) is registered as a view:
SELECT s.name,s.department,d.hod
FROM students s JOIN departments d
ON s.department=d.department;
```

Expected results

```
Students with marks >= 85: Monika (90), Priya (85), Rahul (95). Grouped averages: CSE = 90.0; IT = 81.6667. The Parquet result is re
```

Explanation

A temporary view allows SQL queries over a DataFrame. Parquet is a columnar file format commonly used for analytical data.

Result

Spark SQL queries were executed and the grouped result was saved and read again in Parquet format.

Practical 6: Unix Demos

Aim

Practice basic Unix commands for file and directory management, searching and counting.

Commands

```
pwd
mkdir bigdata
cd bigdata
printf 'big data\nspark data\nbig analytics\n' > sample.txt
ls
cat sample.txt
head -n 2 sample.txt
tail -n 2 sample.txt
grep 'big' sample.txt
grep -c 'big' sample.txt
wc -l sample.txt
wc -w sample.txt
cp sample.txt copy.txt
mv copy.txt renamed.txt
ls -l
```

Pipe example

```
cat sample.txt | grep 'data'
```

Expected output

grep 'big' displays: big data and big analytics. grep -c 'big' displays 2. wc -l displays 3. The pipe displays lines containing data.

Explanation

pwd shows the current directory; mkdir creates a directory; cp copies; mv moves or renames; cat displays a file; head and tail show beginning and end; grep searches; wc counts; | passes output to another command.

Result

Basic Unix commands were executed to create, display, search, copy, move and count file contents.

Practical 7: HDFS Demos

Aim

Create an HDFS directory and upload, list, read, copy, move and delete a file.

Create local sample file

```
echo 'Big Data using Hadoop HDFS' > sample.txt
```

HDFS commands

```
hdfs dfs -mkdir -p /user/student/bigdata
hdfs dfs -put sample.txt /user/student/bigdata/
hdfs dfs -ls /user/student/bigdata
hdfs dfs -cat /user/student/bigdata/sample.txt
hdfs dfs -cp /user/student/bigdata/sample.txt /user/student/bigdata/copy.txt
hdfs dfs -mv /user/student/bigdata/copy.txt /user/student/bigdata/renamed.txt
hdfs dfs -rm /user/student/bigdata/sample.txt
hdfs dfs -rm /user/student/bigdata/renamed.txt
hdfs dfs -rmdir /user/student/bigdata
```

Expected output

The listing shows sample.txt. The cat command displays: Big Data using Hadoop HDFS. After both files are deleted, the empty directory can be removed.

Explanation

HDFS is Hadoop's distributed file system. hdfs dfs commands manage files and directories stored in HDFS. Run these commands only in a configured Hadoop lab.

Result

HDFS directory creation, upload, listing, reading, copy, move and deletion commands were demonstrated.

Practical 8: MapReduce – Word Count

Aim

Run the Hadoop MapReduce WordCount example and understand mapper and reducer stages.

Create input file

```
echo 'big data big spark' > input.txt
echo 'data processing' >> input.txt
```

Upload input and run job

```
hdfs dfs -mkdir -p /user/student/wc_input
hdfs dfs -put -f input.txt /user/student/wc_input/
hdfs dfs -rm -r -f /user/student/wc_output
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount /user/student/wc_input /user/student/wc_out
```

View output

```
hdfs dfs -cat /user/student/wc_output/part-r-00000
```

Expected output (order may vary)

```
big  2
data 2
processing 1
spark 1
```

Explanation

The mapper reads input and emits key-value pairs such as (big,1). Hadoop groups values with the same key. The reducer adds the values for each word to produce the final frequency.

Result

MapReduce WordCount produced the frequency of each word in the input text.

Practical 9: Hive Demos – Basic Queries

Aim

Create a Hive database and table, load CSV data and execute SELECT, WHERE, GROUP BY and ORDER BY.

Sample CSV: sales.csv

```
1,Chennai,Mobile,20000
2,Madurai,Laptop,50000
3,Chennai,Laptop,45000
4,Salem,Mobile,18000
5,Madurai,Tablet,15000
```

Create database and table

```
CREATE DATABASE IF NOT EXISTS salesdb;
USE salesdb;
CREATE TABLE sales (
 sale_id INT, city STRING, product STRING, amount INT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

Load data

```
LOAD DATA LOCAL INPATH '/path/sales.csv' INTO TABLE sales;
```

Queries

```
SELECT * FROM sales;
SELECT * FROM sales WHERE city='Chennai';
SELECT city,SUM(amount) AS total_sales FROM sales GROUP BY city;
SELECT * FROM sales ORDER BY amount DESC;
```

Expected grouped output

```
Chennai  65000
Madurai  65000
Salem    18000
```

Explanation

SELECT displays records; WHERE filters rows; GROUP BY groups rows with the same city; SUM calculates total sales; ORDER BY sorts the result. Compare the output with the original CSV.

Result

Hive database and table were created, CSV data was loaded, and basic queries were executed.

Quick Viva Questions

What is Big Data? Very large and complex datasets that need specialized tools for storage and processing.

What is Hadoop? An open-source framework for distributed storage and processing.

What is HDFS? Hadoop Distributed File System, which stores files across a cluster.

What is Hive? A data warehouse tool that provides SQL-like querying for data stored in Hadoop.

What is Spark? A distributed data-processing engine.

What is an RDD? Resilient Distributed Dataset, a fault-tolerant distributed collection in Spark.

Transformation vs action? A transformation creates a new RDD; an action triggers computation or returns a result.

What is a DataFrame? A distributed collection of structured data organized into named columns.

What is Spark SQL? A Spark module for querying structured data using SQL.

What is MapReduce? A programming model that processes data through Map and Reduce stages.

What is a mapper? The Map stage reads input and emits intermediate key-value pairs.

What is a reducer? The Reduce stage combines values for the same key.

What is Hive partitioning? Organizing table data into separate partitions based on column values.

What is bucketing? Dividing table rows into a fixed number of files using a hash of a column.

What is grep? A Unix command used to search text matching a pattern.

What is a pipe? The | operator sends the output of one command as input to another.

Submission reminder: Include commands/programs and screenshots of your actual lab outputs. Paths, usernames, Spark versions and Hadoop JAR locations may differ in your institution.